МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
КАФЕДРА САПР

Практическая работа №1

по дисциплине

«Объектно-ориентированное программирование»

Студент гр. 5354 _____ Головко Е.Д.

Преподаватель _____ Кулагин М.В.

Санкт-Петербург
2026

1. Задание

Разработайте консольное приложение, которое моделирует работу небольшой службы доставки.

Программа работает только в памяти: внешние файлы, база данных, сетевые запросы и графический интерфейс не являются обязательными.

Пользователь должен иметь возможность создать заказ, выбрать способ доставки, назначить курьера, просмотреть список заказов и переводить заказ по допустимым статусам. В программе должны быть предусмотрены корректная обработка ошибочного ввода и понятные сообщения для пользователя.

Предметная область

Минимальная модель программы должна включать:

1) заказ - номер, данные клиента, адрес доставки, список позиций, стоимость, способ доставки, назначенный курьер и текущий статус;

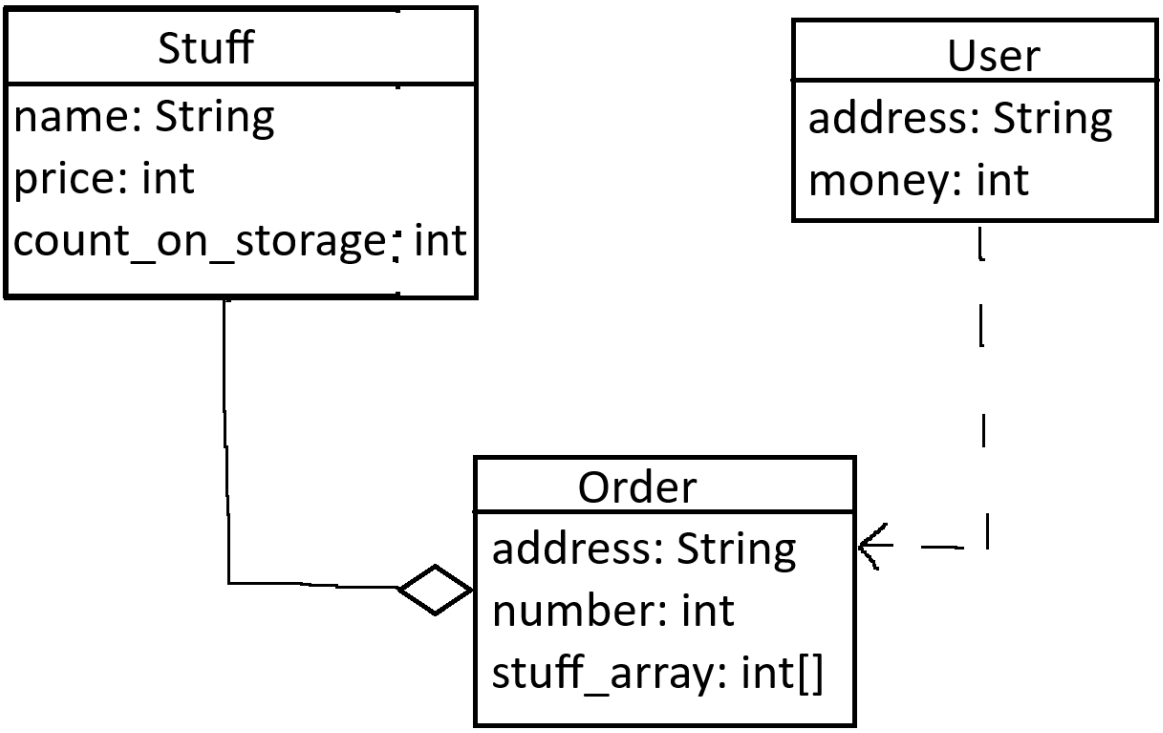
2) клиента и курьера как отдельные сущности с необходимыми полями и поведением;

3) позицию заказа - название товара, количество и цену;

4) общий контракт для способа доставки - например, интерфейс DeliveryMethod или абстрактный класс;

5) не менее трёх способов доставки: стандартная доставка курьером, экспресс-доставка и самовывоз

2. UML–диаграмма.



Пунктирная стрелка от User до Order означает, что при изменении поля address у объекта изменяется и address в Order (зависимость). Сплошная линия с ромбом на конце означает, что класс Stuff является частью Order (агрегация по-умному).

3. Описание интерфейса пользователя программы

Пользователь выбирает пункты из предложенных на экране (в консоли). В консоли выводятся команды выбора товара, доставки, выбора курьера и статус доставки.

Примеры:

```
Здравствуйте! Приветствуем вас в нашем магазине "Lowkey"
Ассортимент:
1. Киндер Пингви  -  40грн
2. Верёвка  -  80грн
3. Мыло  -  70грн
4. Тёмные души 3 игра на русском языке  -  500грн
5. Портрет главы ДНР Д. Пушилина  -  400грн
6. Фигурка ФанкоПоп Беззубик из " Как приручить дракона"  -  1000грн
Что хотите заказать? Введите номер товара (для завершения совершения заказа введите '0', для отмены - '-1'):
4
Введите кол-во товара:
2
Ваш заказ: Тёмные души 3 игра на русском языке x 2;
Общая сумма заказа: 1000
Ассортимент:
1. Киндер Пингви  -  40грн
2. Верёвка  -  80грн
3. Мыло  -  70грн
4. Тёмные души 3 игра на русском языке  -  500грн
5. Портрет главы ДНР Д. Пушилина  -  400грн
6. Фигурка ФанкоПоп Беззубик из " Как приручить дракона"  -  1000грн
Что хотите заказать? Введите номер товара (для завершения совершения заказа введите '0', для отмены - '-1'):
0
Ваш заказ: Тёмные души 3 игра на русском языке x 2;
Общая сумма заказа: 1000
Как хотите получить заказ?
1 Самовывоз;
2 Доставкой;
3 Экспресс-доставкой;
0 Вернуться.
3
Введите адрес доставки:
asdsadasdsa
Выберите курьера:
1.  Мухридин;
2.  Рахмаджон;
3.  Мансур.
3
```

```
Выберите курьера:
1.  Мухридин;
2.  Рахмаджон;
3.  Мансур.
3
С вас 1108 гривен.
Вам везёт заказ Мансур.
Ожидайте заказ номер: 2. Осталось 1 минут.
Ожидайте заказ номер: 2. Осталось 0 минут.
Заберите заказ!
История заказов:

N2: Тёмные души 3 игра на русском языке X 2,
Ассортимент:
1. Киндер Пингви  -  40грн
2. Верёвка  -  80грн
3. Мыло  -  70грн
4. Тёмные души 3 игра на русском языке  -  500грн
5. Портрет главы ДНР Д. Пушилина  -  400грн
6. Фигурка ФанкоПоп Беззубик из " Как приручить дракона"  -  1000грн
Что хотите заказать? Введите номер товара (для завершения совершения заказа введите '0', для отмены - '-1'):
```

4. Текст программы

```
import  java.util.Scanner;

class Stuff
{
   String name;
   int price;
   int count_on_storage;
   public Stuff (String name, int price, int count_on_storage)
   {
      this.name = name;
      this.price = price;
      this.count_on_storage = count_on_storage;
   }
}
class User{
   String adress;
   int money;
   public User (String adress, int money)
   {
      this.adress = adress;
      this.money = money;
   }
}
class Order{
   String adress;
   int number;
   int[] stuff_array = new int[list.length];
   public Order (String adress, int number, int[] stuff_array){
      this.adress = adress;
      this.number = number;
      for (int i = 0; i < stuff_array.length; i++)
         this.stuff_array[i] = stuff_array[i];
   }
}
User user = new User("случайный адрес", 5000 );
Stuff kinder_pingie = new Stuff ("Киндер Пингви" , 40, 10);
Stuff rope = new Stuff ("Верёвка", 80, 5);
Stuff soap = new Stuff ("Мыло", 70, 15);
Stuff dark_souls_3 = new Stuff("Тёмные души 3 игра на русском языке", 500, 3);
Stuff pushilins_portrait = new Stuff ("Портрет главы ДНР Д. Пушилина", 400, 1);
Stuff toothless_funkopop_figure = new Stuff ("Фигурка ФанкоПоп Беззубик из" +
      " \" Как приручить дракона\" ", 1000, 3);
String hello_phrase = "Здравствуйте! Приветствуем вас в нашем магазине \"Lowkey\" \n";
Scanner scanner = new Scanner(System.in);
int user_choice;
Stuff[] list = {kinder_pingie, rope, soap, dark_souls_3, pushilins_portrait, toothless_funkopop_figure};
int count_of_orders = 0;

final int exit_code = 0;
```

```
final int return_stuff_code = -1;
final int samovivoz_code = 1, delivery_code = 2, fast_delivery_code=3;
final double delivery_koef = 1.2, fast_delivery_koef = 1.5;
int trail_long =(int) (Math.random()*20) + 1;
final int courier_speed = 2, fast_courier_speed = 3;
int[] speeds = {courier_speed, fast_courier_speed};
String delivery_adress = "ул. Грибоедова д.6";
int user_delivery_choice, user_courier_choice;
boolean order_empty = true;
boolean work_done = false;
Order[] list_of_orders = new Order[10];
int[] order = new int[list.length];
long total_sum = 0;
String[] couriers = {"Мухридин", "Рахмаджон", "Мансур"};
int order_number;
int[] array_in_zero(int[] a)
{
   for (int i = 0; i < a.length; i++)
      a[i] = 0;
   return a;
}
int check_enter ( int low_border, int high_border)
{
   int message_int = 0;
   String message;
   boolean wrong_enter = false;
   do {
      message = scanner.next();
      wrong_enter = false;
      try {
         message_int = Integer.parseInt(message);
         if (message_int <= low_border || message_int > high_border)
            System.out.print("Неверный ввод!" + '\n');
      } catch (NumberFormatException e) {
         System.out.print("Неправильный ввод!" + '\n');
         wrong_enter = true;
      }
   } while (wrong_enter || message_int <= low_border || message_int > high_border);
   return message_int;
}
void delete_stuff()
{
   int index_delete_stuff;
   int count_delete_stuff;
   System.out.print("Какой товар хотите вернуть? Введите индекс товара: \n");
   index_delete_stuff = check_enter(0, list.length)  - 1;
   if (order[index_delete_stuff] > 0) {
      System.out.print("Какое количество товара хотите вернуть? \n");
      count_delete_stuff = check_enter(-1, order[index_delete_stuff]);
      if (count_delete_stuff <= order[index_delete_stuff]){
         order[index_delete_stuff]-= count_delete_stuff;
         list[index_delete_stuff].count_on_storage += count_delete_stuff;
```

```
            total_sum-= count_delete_stuff * list[index_delete_stuff].price;
        }
        else
            System.out.print("В корзине нет столько товара! \n");
    }
    else
        System.out.print("В корзине нет товара по этому индексу! \n");
    order_empty = true;
    for (int i = 0; i < list.length; i++ ) // Проверяем, не опустел ли заказ
        if (order[i]>0) order_empty=false;

}
void menu_of_stuff(int stuff_number)
{
    System.out.print("Введите кол-во товара: " + '\n');
    int count_stuff = scanner.nextInt();
    if (count_stuff <= list[stuff_number-1].count_on_storage ){
        order[stuff_number-1] = count_stuff;
        list[stuff_number-1].count_on_storage = list[stuff_number-1].count_on_storage - count_stuff;
        if (count_stuff>0)
            order_empty = false;
        total_sum += list[stuff_number-1].price * count_stuff;
    }
    else
        System.out.print("Операция не может быть выполнена"+ '\n');

}
void delivery_menu() {
    System.out.print("Как хотите получить заказ?" + '\n' + samovivoz_code + " Самовывоз;" + '\n' +
delivery_code
            + " Доставкой;" + '\n' + fast_delivery_code + " Экспресс-доставкой;" + '\n' + exit_code + " Вернуться.
\n");
    user_delivery_choice = check_enter(exit_code - 1, fast_delivery_code);
    switch (user_delivery_choice) {
        case exit_code:
            break;
        case samovivoz_code:
            break;
        case delivery_code:
            System.out.print("Введите адрес доставки:" + '\n');
            user.adress = scanner.next();
            total_sum = Math.round(total_sum + delivery_koef * trail_long) + 100;
            break;
        case fast_delivery_code:
            System.out.print("Введите адрес доставки:" + '\n');
            user.adress = scanner.next();
            total_sum = Math.round(total_sum + trail_long * fast_delivery_koef) + 100;
    }
    if (user_delivery_choice != samovivoz_code && user_delivery_choice != exit_code) {
        System.out.print("Выберите курьера: " + '\n');
        for (int i = 0; i < couriers.length; i++) {
            if (i != couriers.length - 1)
```

```
            System.out.print(i + 1 + ". " + ' ' + couriers[i] + ';' + '\n');
        else
            System.out.print(i + 1 + ". " + ' ' + couriers[i] + '.' + '\n');
    }
    user_courier_choice = check_enter(0, couriers.length);
    System.out.print("С вас " + total_sum + " гривен." + '\n');
    if (user.money>=total_sum) {
        System.out.print("Вам везёт заказ " + couriers[user_courier_choice - 1] + '.' + '\n');
        while (trail_long > 0) {
            System.out.print("Ожидайте заказ номер: " + order_number + ". Осталось " + trail_long /
speeds[user_delivery_choice - 2] + " минут. \n");
            trail_long -= speeds[user_delivery_choice - 2];
        }
        System.out.print("Заберите заказ! \n");
        list_of_orders[count_of_orders] = new Order(user.adress, order_number,order);
        count_of_orders++;
        order_empty = true; // Для оформления нового заказа
        array_in_zero(order);
        user.money-= total_sum;
        total_sum = 0;
    }
    else
        System.out.print("Недостаточно средств. \n");
  } else if (user_delivery_choice == samovivoz_code) {
    System.out.print("С вас " + total_sum + " гривен." + '\n');
    if (user.money>=total_sum) {
        System.out.print("Номер вашего заказа: " + order_number + ". Заберите заказ по адресу: " +
delivery_adress + '.' + '\n');
        order_empty = true; // Для оформления нового заказа
        list_of_orders[count_of_orders] = new Order(delivery_adress, order_number,order);
        count_of_orders++;
        array_in_zero(order);
        user.money-=total_sum;
        total_sum = 0;
    }
    else
        System.out.print("Недостаточно средств. \n");
  }
}
void main()
{
  System.out.print(hello_phrase);

  do {
    order_number = (int) (Math.random() * 20);
    if (count_of_orders>0) {
        System.out.print("История заказов: \n ");
        for (int i = 0; i < count_of_orders; i++) {

            System.out.print('\n'+ "N" + list_of_orders[i].number + ':');
            for (int j = 0; j < list.length; j++)
                if (list_of_orders[i].stuff_array[j] > 0)
```

```
                System.out.print(' '+list[j].name + " X " + list_of_orders[i].stuff_array[j] + ',');
            }
            System.out.print('\n');
        }

        do {
            System.out.print("Ассортимент:" + '\n');
            for (int i = 0; i < list.length; i++) {
                if (list[i].count_on_storage > 0)
                    System.out.print((i + 1) + ". " + list[i].name + "  -  " + list[i].price + "грн" + '\n');
            }
            System.out.print("Что хотите заказать? Введите номер товара (для завершения совершения заказа
введите '0', для отмены - '-1'):" + '\n');
            user_choice = check_enter(return_stuff_code - 1, list.length);
            if (user_choice > exit_code && user_choice <= list.length)
                menu_of_stuff(user_choice);
            else if (user_choice == return_stuff_code && !order_empty)
                delete_stuff();
            if (!order_empty) {
                System.out.print("Ваш заказ:");
                for (int i = 0; i < order.length; i++)
                    if (order[i] != 0)
                        System.out.print(' ' + (list[i].name) + " x " + order[i] + ';');
                System.out.print('\n' + "Общая сумма заказа: " + total_sum + '\n');
            }
        } while (user_choice != exit_code);
        if (!order_empty)  // Если заказ не пуст
            delivery_menu();
        else
            work_done = true;
    }while (!work_done);
    System.out.print("Допобачення!" + '\n');
}
```

5. Выводы

На этой практической работе получилось создать консольное приложение для пользования доставкой.

Разработанный программный код собирался с помощью встроенной системы автоматизированной сборки Inteliji IDEA. Результаты были выложены на Github:

https://github.com/evor18/Practical-W-1-OOP.git